

Learning to Combine Online Algorithms with Predictions Online

Senem Işık

Ice-breaker

Behavioral Biases, Time-Inconsistency, and Social Media Use

Jon Kleinberg

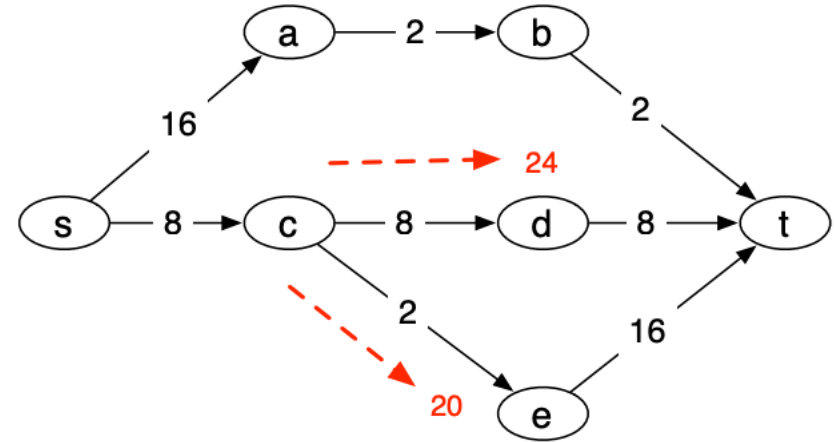
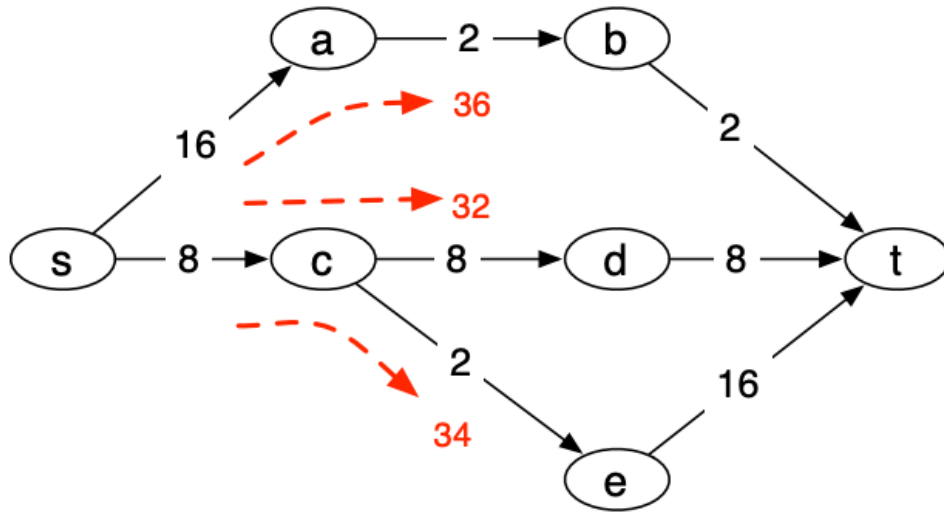
Including joint work with Sendhil Mullainathan, Manish Raghavan, and Sigal Oren.

Cornell University



Behavioral Biases,
Time-Inconsistency,
and
Senem needs to start cooking.

A graph theoretic framework

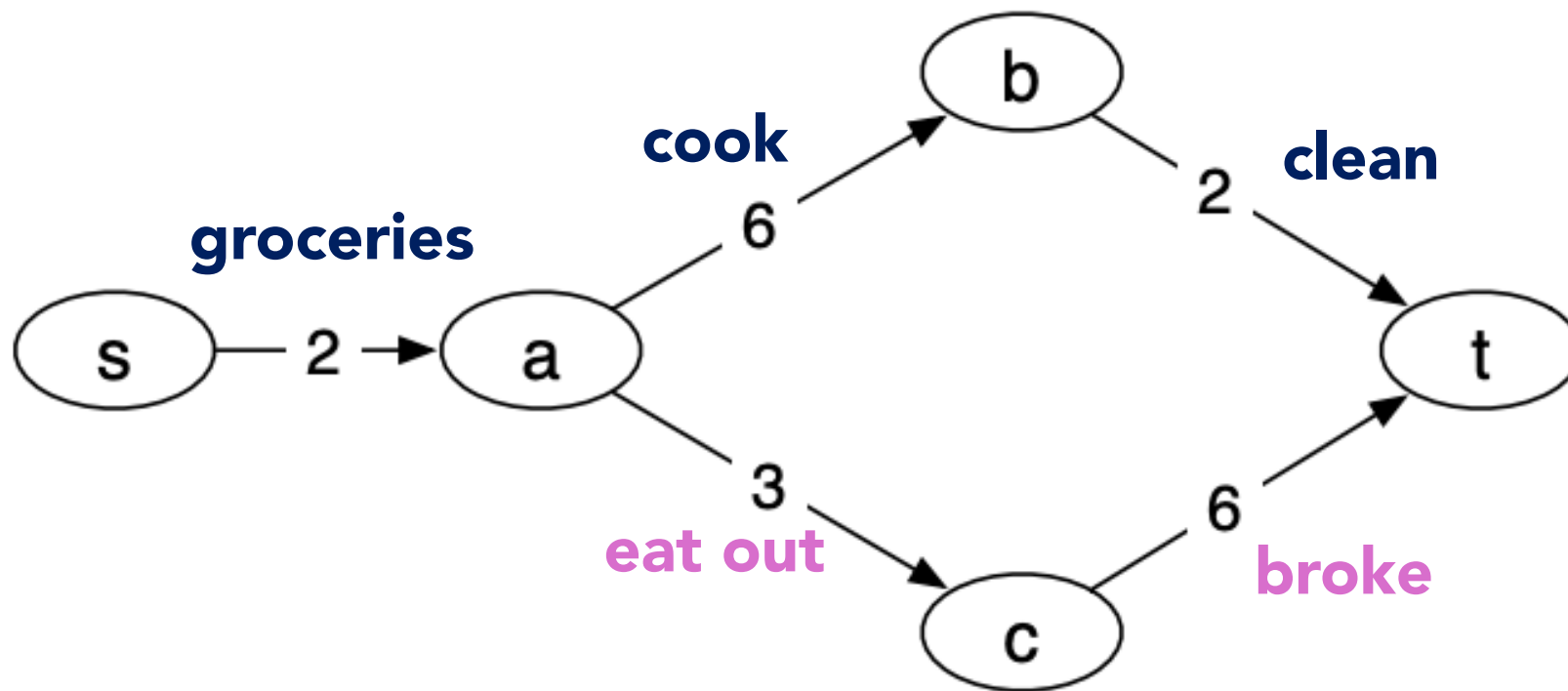


Use graphs as basic structure to represent scenarios

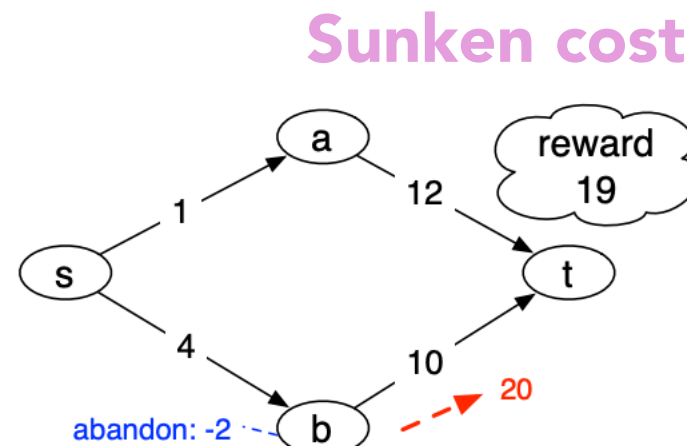
[Kleinberg-Oren 2014]

- Agent plans to follow cheapest path from s to t .
- From a given node, immediately outgoing edges have costs multiplied by $b > 1$.

Senem's cooking story



What should Senem do?



What is my sunken cost?

Learning to Combine Online Algorithms with Predictions Online

Senem Işık

Online Algorithms with Predictions

An online problem is one where inputs arrive **one by one** over time, and you must make decisions **immediately** without knowing future inputs (often with **irreversible** choices).

Case study: Optimal Stopping



Online Algorithms with Predictions

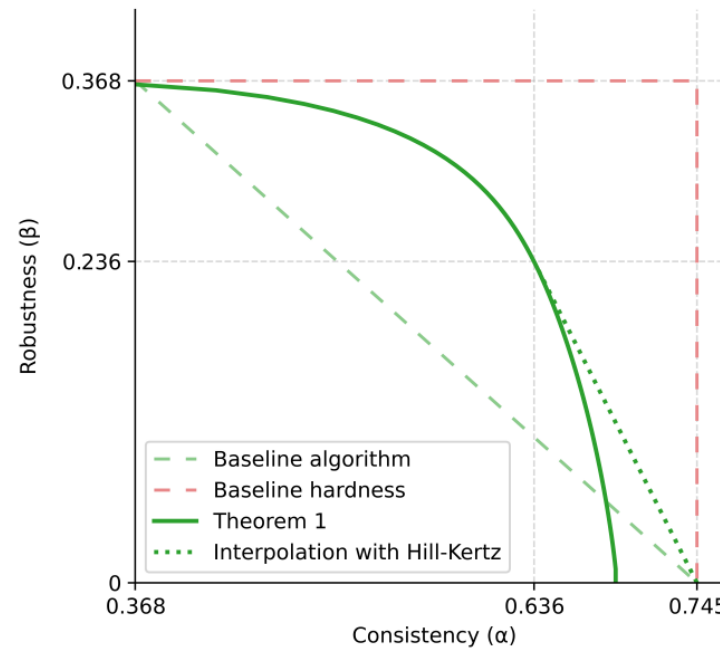
Case study: Optimal Stopping with a Predicted Prior (Bai et. al.)

Each point in this curve correspond to a different consistency-robustness tradeoff

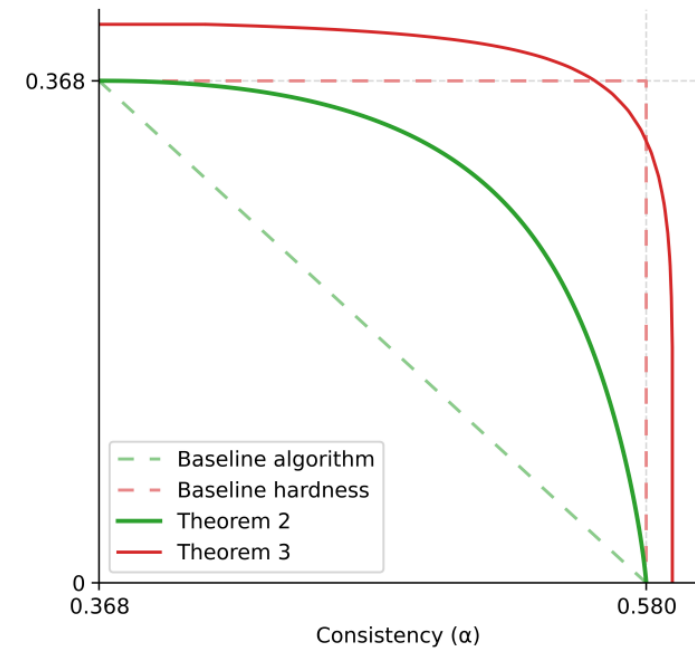
Question: What is the ideal trade-off?

Classic answer:

Depends on how much you trust your prediction... You can approximate the ideal point based on your dataset.



(a) MaxExp



(b) MaxProb

Uncertainty quantification

Online Algorithms with Predictions

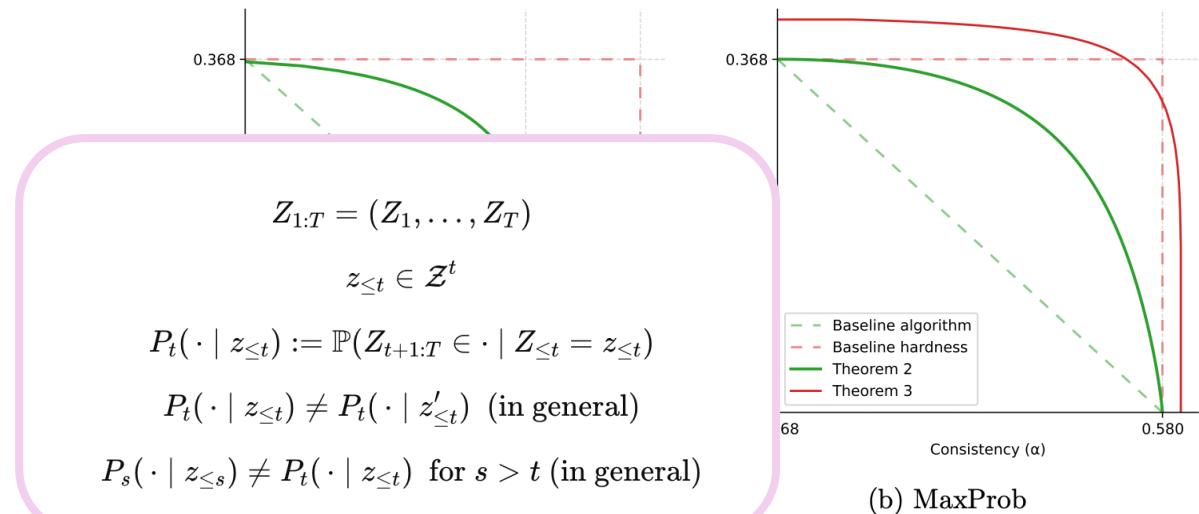
Case study: Optimal Stopping with a Predicted Prior (Bai et. al., 2025)

What if?

- (1) There is a **distribution shift** between the training dataset and online deployment.
- (2) Even if there is **no** distribution shift:

If we had a way to set this trade-off **adaptively per instance**, this would be better than

a **fixed** robustness-consistency tradeoff that may work well for a distribution on average.



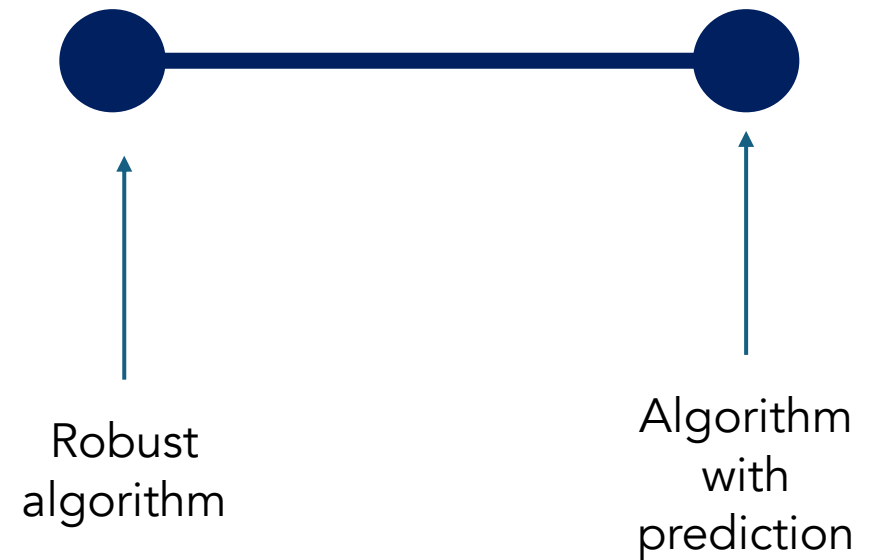
“Our framework is limited in that it does not adjust thresholds adaptively based on the observed values. Intuitively, a “learning” algorithm may adjust thresholds toward [robust algorithm] if the observed values are unlikely to be from the predicted prior, and conversely, toward the [consistent] algorithm [...] if the observed values are consistent with the predicted prior.”

More generally

- **Predictions = configurations** can be anything:
 - from what weights you start your neural network
 - to the literal solution of your problem...

We want to characterize how to optimally **combine**:

- Different candidates of robustness-consistency tradeoff if you have **one** prediction, or
- A **portfolio** of predictions
 - each of them predicting different or the same aspect of problem with varying levels of accuracy and confidence

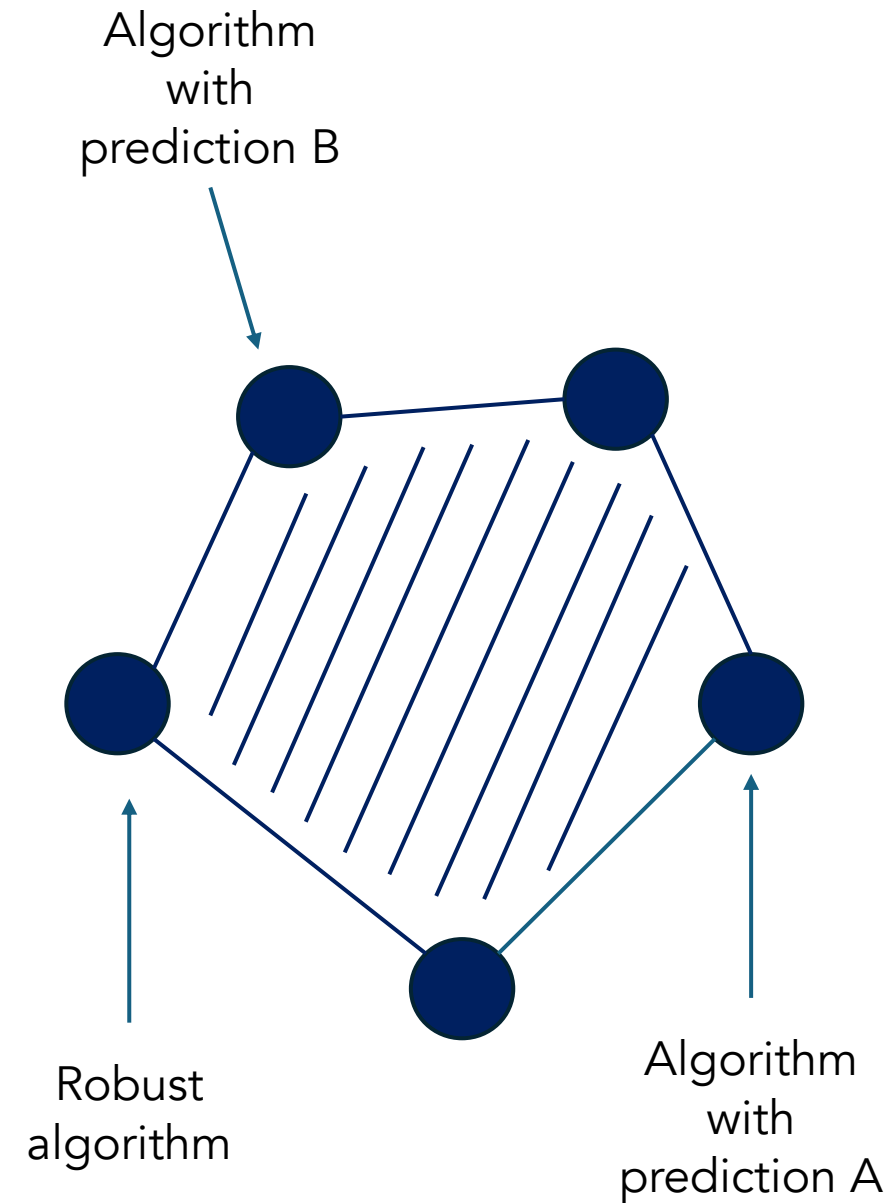


More generally

- **Predictions = configurations** can be anything:
 - from what weights you start your neural network
 - to the literal solution of your problem...

We want to characterize how to optimally **choose/combine**:

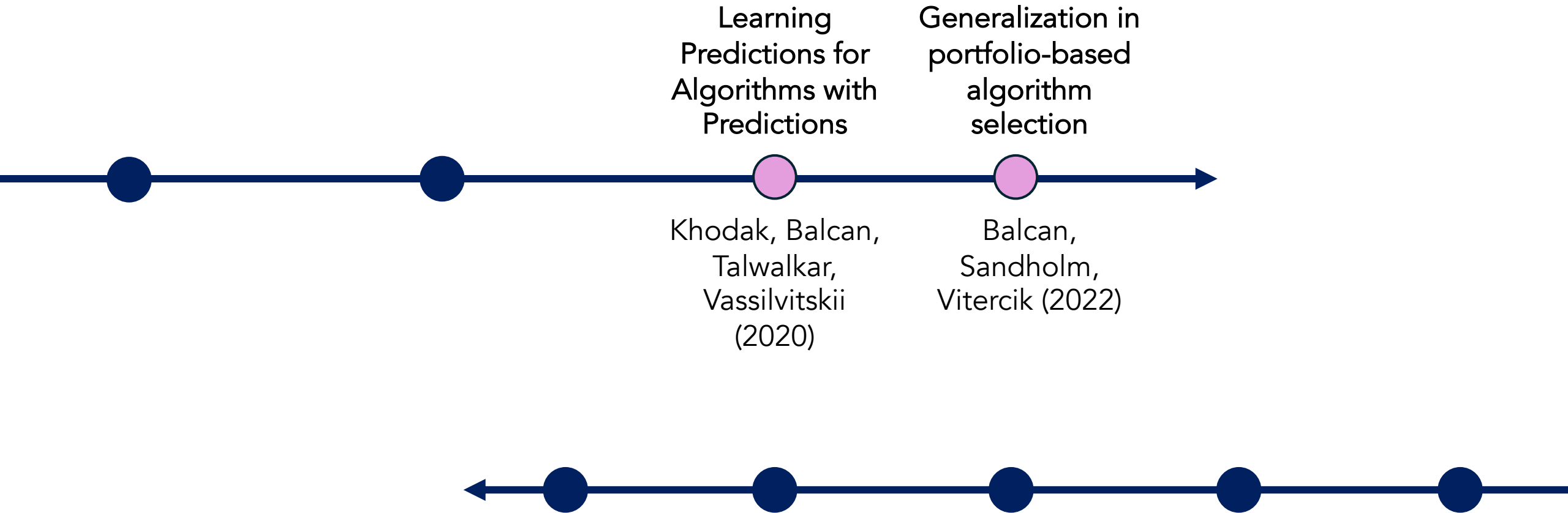
- Different candidates of robustness-consistency tradeoff if you have **one** prediction, or
- A **portfolio** of predictions
 - each of them predicting different or the same aspect of problem with varying levels of accuracy and confidence



How can **online samples** during deployment help us tune our algorithms online?

kind of **warm start** but continuously do it

Prior work



Learning Predictions for Algorithms with Predictions

1. For a given algorithm, derive a convenient-to-optimize **upper bound** $U_t(x)$ on the **cost** $C_t(x)$ that depends on both the prediction x and information specific to instance t returned once the algorithm terminates. (They find that in many cases such bounds already exist, and the quality of the prediction can be measured by a distance from some ground truth obtained from the output, a quantity that is usually convex and thus learnable.)
2. Apply online learning to obtain both **regret guarantees** against adversarial sequences and **sample complexity** bounds for i.i.d. instances

Algorithm 1: Generic application of an online learning algorithm over $\mathcal{X}$ to learn a predictor for a method `AlgorithmWithPrediction` that takes advice from $\mathcal{X}$ and returns upper bounds U_t on its cost. The goal of `OnlineAlgorithm` is low regret over the sequence U_t , so that on-average C_t is upper bounded by the smallest possible average of U_t , up to some error decreasing in T . For specific instantiations of algorithms and feedback see Table 1.

```
initialize  $\mathbf{x}_1 \in \mathcal{X}$  for OnlineAlgorithm
for instance  $t = 1, \dots, T$  do
    obtain instance  $I_t$ 
    run AlgorithmWithPrediction( $I_t, \mathbf{x}_t$ )
        suffer cost  $C_t(\mathbf{x}_t) \leq U_t(\mathbf{x}_t)$ 
        get feedback to construct upper bound  $U_t$ 
     $\mathbf{x}_{t+1} \leftarrow \text{OnlineAlgorithm}(\{U_t\}_{s=1}^t, \mathbf{x}_1)$ 
```

Learning Predictions for Algorithms with Predictions

1. Bipartite matching (prediction: the duals of the primal-dual algorithm, online algo: projected online sub-gradient descent)
2. Online page migration (prediction = the sequence of requests, online algo: exponential subgradient)
3. Learning linear predictors with instance feature inputs (prediction = the linear predictor, or the weights of the predictor)
4. Robust consistency trade-off
 - For ski-rental, they learn the prediction jointly with the tradeoff

For online problems, you don't know the instance *a priori*. At any point, you have partial information → you should adapt as the problem reveals itself.

They wait until the algorithm terminates,
and are learning a **fixed, non-adaptive prediction** for the whole instance.

```
initialize  $\mathbf{x}_1 \in \mathcal{X}$  for OnlineAlgorithm
for instance  $t = 1, \dots, T$  do
    obtain instance  $I_t$ 
    run AlgorithmWithPrediction( $I_t, \mathbf{x}_t$ )
    suffer cost  $C_t(\mathbf{x}_t) \leq U_t(\mathbf{x}_t)$ 
    get feedback to construct upper bound  $U_t$ 
     $\mathbf{x}_{t+1} \leftarrow \text{OnlineAlgorithm}(\{U_t\}_{s=1}^t, \mathbf{x}_1)$ 
```

Generalization in portfolio-based algorithm selection

Problem formulation. A portfolio-based algorithm selection procedure relies on two key components: a *portfolio* and an *algorithm selector*. A portfolio is a set $\mathcal{P} = \{\rho_1, \dots, \rho_\kappa\} \subseteq \mathbb{R}$ of κ parameter settings. An algorithm selector is a mapping $f : \mathcal{Z} \rightarrow \mathcal{P}$ from problem instances $z \in \mathcal{Z}$ to parameter settings $f(z) \in \mathcal{P}$. In practice [22, 32, 39], the portfolio and algorithm selector are typically learned using the following high-level procedure:

1. Choose a class $\mathcal{F}$ of algorithm selectors, each of which maps $\mathcal{Z}$ to $\mathbb{R}$. (In Section 4, we provide several examples of classes $\mathcal{F}$ used in practice.)
2. Draw a training set $\mathcal{S} = \{z_1, \dots, z_N\} \sim \mathcal{D}^N$ of problem instances from the unknown distribution $\mathcal{D}$.
3. Use $\mathcal{S}$ to learn a portfolio $\hat{\mathcal{P}} = \{\rho_1, \dots, \rho_\kappa\} \subseteq \mathbb{R}$.
4. Use $\mathcal{S}$ to learn an algorithm selector $\hat{f} \in \mathcal{F}$ that maps to parameter settings in the portfolio $\hat{\mathcal{P}}$.

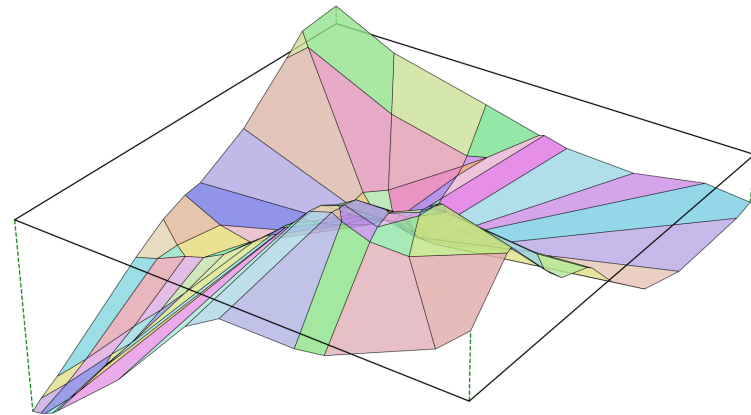
Generalization in portfolio-based algorithm selection

When algorithmic performance is a **piecewise-constant function of the prediction**, they give a **sample complexity** of learning a predictor $\hat{f}(\cdot)$ such that with probability $1 - \delta$,

$$\max_{\mathcal{P}: |\mathcal{P}| \leq \kappa} \mathbb{E}_{z \sim \mathcal{D}} \left[\max_{p \in \mathcal{P}} u_p(z) \right] - \mathbb{E}_{z \sim \mathcal{D}} [u_{\hat{f}(z)}(z)] < \varepsilon.$$

This depends on the intrinsic complexity of the

- selector
 - Regression trees
 - Clustering based algorithms
- number of pieces
- size of the portfolio κ



Generalization in portfolio-based algorithm selection

Problem formulation. A portfolio-based algorithm selection procedure relies on two key components: a *portfolio* and an *algorithm selector*. A portfolio is a set $\mathcal{P} = \{\rho_1, \dots, \rho_\kappa\} \subseteq \mathbb{R}$ of κ parameter settings. An algorithm selector is a mapping $f: \mathcal{Z} \rightarrow \mathcal{P}$ that maps problem instances $z \in \mathcal{Z}$ to parameter settings $f(z) \in \mathcal{P}$. In practice [22, 3] f is typically learned using the following high-level procedure.

1. Choose a class $\mathcal{F}$ of algorithm selectors, each of which maps problem instances to parameter settings. In practice, several examples of classes $\mathcal{F}$ used in practice.
2. Draw a training set $\mathcal{S} = \{z_1, \dots, z_N\} \sim \mathcal{D}^N$ of problem instances from the unknown distribution $\mathcal{D}$.
3. Use $\mathcal{S}$ to learn a portfolio $\hat{\mathcal{P}} = \{\rho_1, \dots, \rho_\kappa\} \subseteq \mathbb{R}$.
4. Use $\mathcal{S}$ to learn an algorithm selector $\hat{f} \in \mathcal{F}$ that maps to parameter settings in the portfolio $\hat{\mathcal{P}}$.

For online problems, you don't know the instance *a priori*. At any point, you have partial information $\rightarrow$ you should adapt as the problem reveals itself.



Machine Learning, 39, 35–58, 2000.
© 2000 Kluwer Academic Publishers. Printed in The Netherlands.

On-line Learning and the Metrical Task System Problem

AVRIM BLUM
CARL BURCH

Department of Computer Science, Carnegie Mellon University, Pittsburgh PA 15213, USA

avrim+@cs.cmu.edu
cburch+@cs.cmu.edu

Editor: Robert Schapire

On-line choice of on-line algorithms

Authors:  [Yossi Azar](#),  [Andrei Z. Broder](#),  [Mark S. Manasse](#) | [Authors Info & Claims](#)

[SODA '93: Proceedings of the fourth annual ACM-SIAM symposium on Discrete algorithms](#) • Pages 432 - 440

Published: 01 January 1993 [Publication History](#)



Online Choice of Online Algorithms

Metrical Task System (MTS)

is an online model where

- an algorithm moves among states in X in a **metric space**,
- each incoming task $\tau_1, \tau_2, \dots$ assigns a **cost** $\tau_t: X \rightarrow \mathbb{R}_{\geq 0}$ to each state,
- the algorithm must choose a state to serve it, paying **service cost** for the chosen state plus **movement cost** equal to the metric distance moved

$$d: X \times X \rightarrow \mathbb{R}_{\geq 0}$$

giving

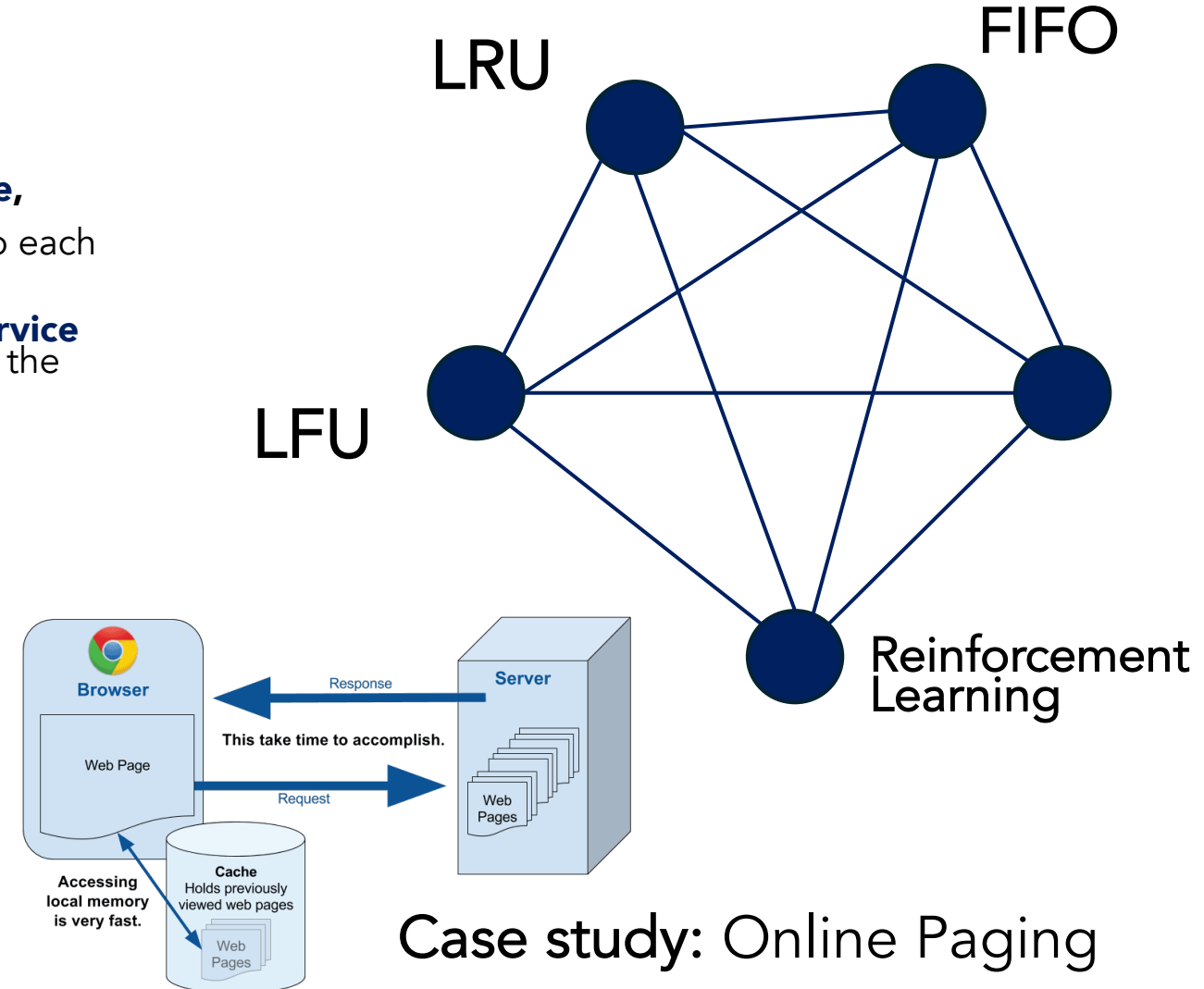
$$\sum_{t=1}^T d(x_{t-1}, x_t) + \tau_t(x_t)$$

as the total cost.

This version has **one-step lookahead**.

Zero-step lookahead version **without the movement cost** is the **Experts-DTF problem**.

One can also have the state of the algorithm as **distribution** over the states.

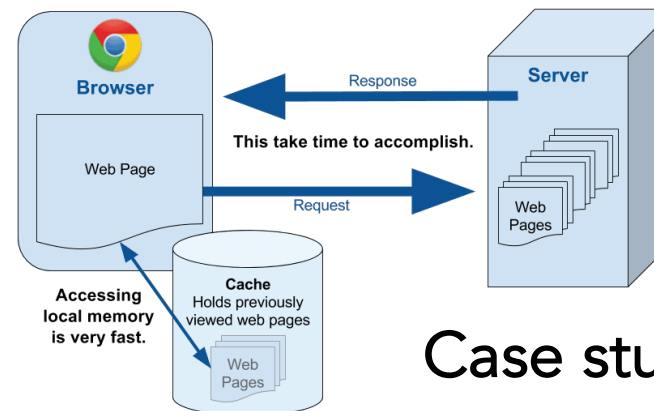
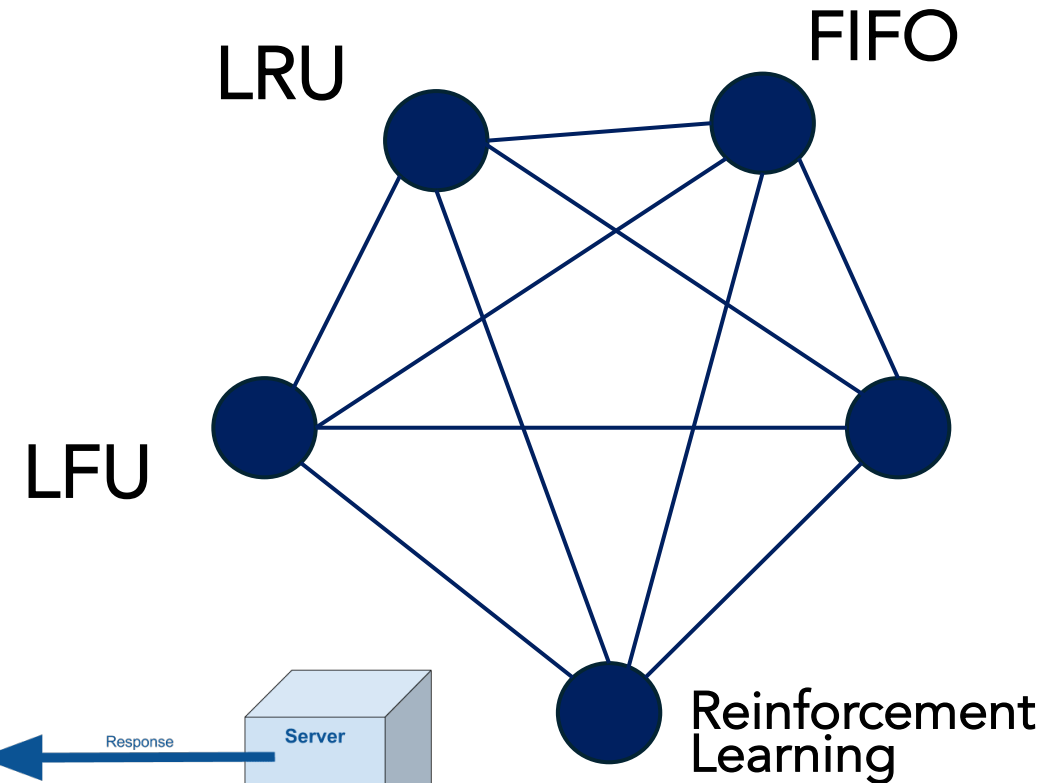


Online Algorithms as “Experts”

Suppose **we do not know** a priori which of these will perform best on an upcoming sequence of requests.

Can we combine these algorithms in some generic way into an on-line strategy whose performance will never be too much worse than the best of them in hindsight?

(Competitive ratio)



Case study: Online Caching

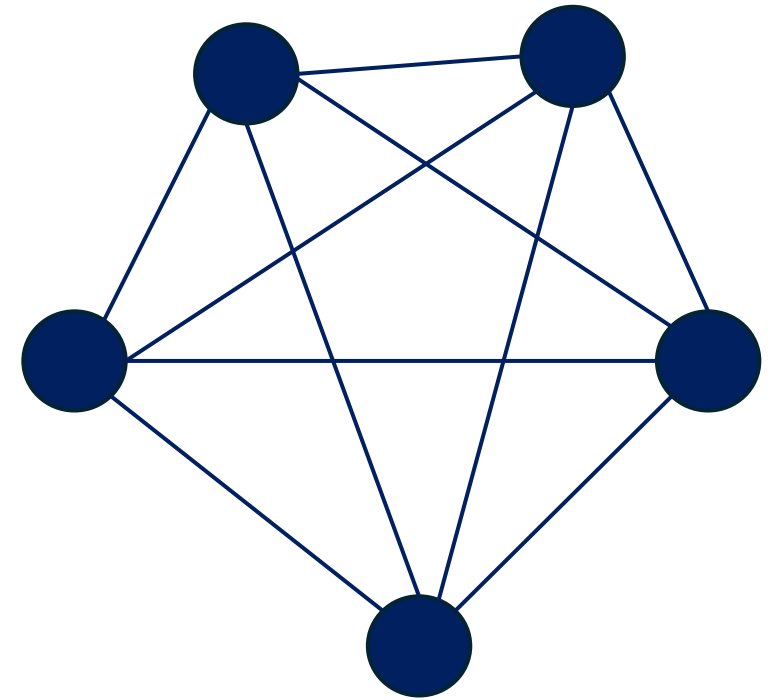
Online Learning and MTS

Hedge (WMR) Algorithm

Initialize $\beta \in [0,1], w_1, w_2, \dots = 1$.

For each time step:

1. Sample an expert i with probability $\frac{w_i}{\sum_{j=1}^n w_j}$
2. Receive the loss ℓ
3. Reset $w_i = w_i \beta^{\ell_i}$



They assumed costs are unit.

Theorem 2 (Littlestone & Warmuth, 1994; Freund & Schapire, 1997). *If the best expert has total loss L ,*

$\frac{\ln(1/\beta)}{L}$

Theorem 3. *The WMR algorithm has the property that if it has probability distribution $\mathbf{p}$, receives loss vector ℓ , and then updates to probability distribution $\mathbf{p}'$, then*

$$d(\mathbf{p}, \mathbf{p}') \leq \ln(1/\beta) \mathbf{p} \cdot \ell.$$

Prior work

Assume a portfolio of predictions

(each corresponds to a sequence of requests
→ how long it will take the most recent request
will be repeated)
with one of them having sublinear regret.

Following (Blum and Burch, 2000) guarantees
doing nearly as well → vanishing regret.

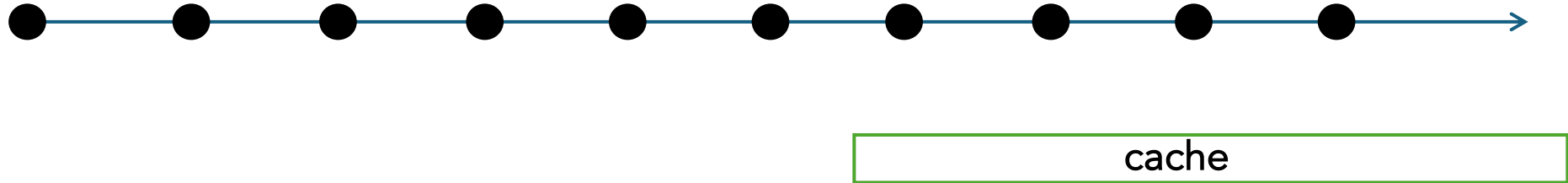


Online Paging
with a Vanishing
Regret

The diagram consists of two horizontal timelines. The top timeline has four dark blue circular markers and an arrow pointing to the right. The bottom timeline has four pink circular markers and an arrow pointing to the left. A light blue speech bubble originates from the third marker on the top timeline and points towards the text 'Online Paging with a Vanishing Regret'. The timeline for 'Online Paging with a Vanishing Regret' starts with a pink marker, followed by three more pink markers further to the right, indicating a progression over time.

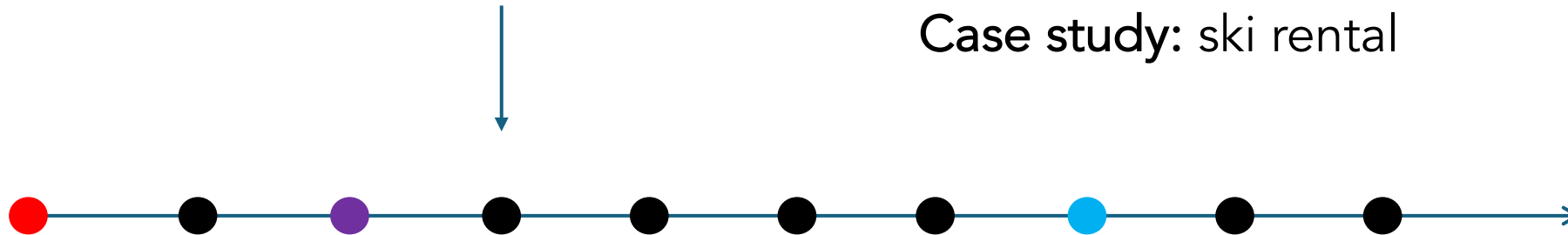
Emek, Kutten,
Shi
(2020)

What if your problem is not quite like
a metrical task problem?



What if your problem is not quite like a metrical task problem?

Case study: ski rental



Algorithm configuration **1**: Rent until day 3

Algorithm configuration **2**: Buy on day 1

Algorithm configuration **3**: Rent until day 8

By the time you have information, it is already too late...

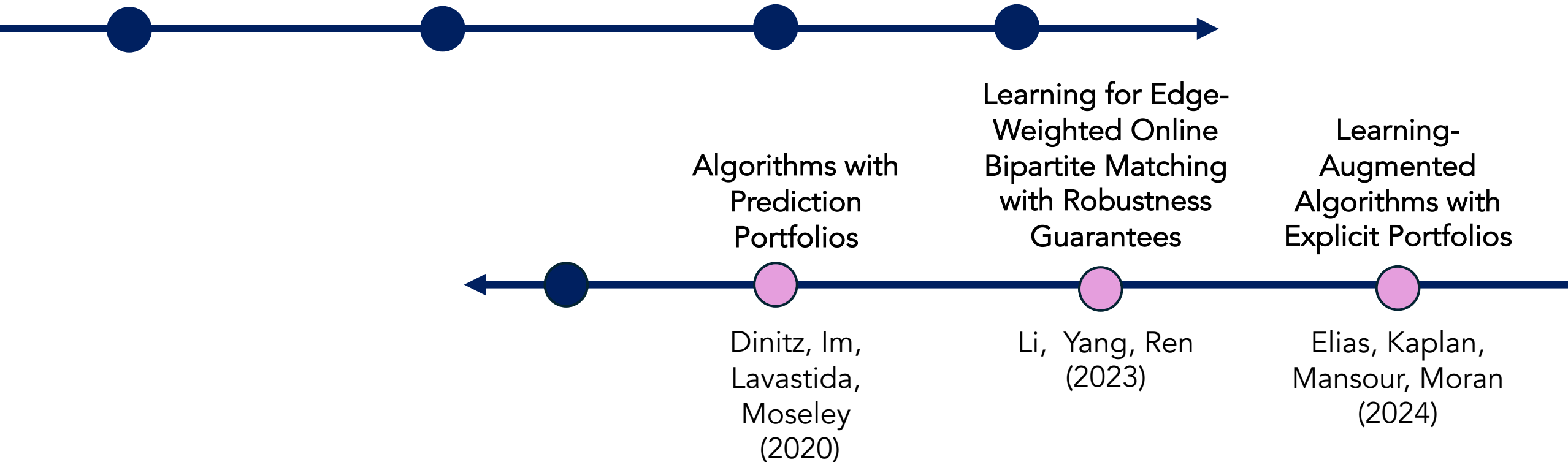
The learning affects decision-making but decision making also constraint learning.

You don't get daily, time-step wise loss!

Your loss looks more like all (you win) or nothing (or game terminates)

Case study 2: Optimal Stopping

Prior work



Algorithms with Prediction Portfolios

1. Minimum Cost Bipartite Matching with Predicted Duals
2. Online Load Balancing with Predicted Machine Weights
3. Scheduling with Predicted Permutations

In contrast, we make the selection during the course of the algorithm, and sometimes switch back and forth while honing in on the best predictor.

Algorithms with Prediction Portfolios

1. **Minimum Cost Bipartite Matching with Predicted Duals**
2. Online Load Balancing with Predicted Machine Weights
3. Scheduling with Predicted Permutations

Given a graph $G = (V, E)$ with edge costs $c \in \mathbb{Z}^E$, find the perfect matching that minimizes the total cost.

The objective is to minimize the runtime.

Algorithm 2 Minimum cost matching with k predicted dual solutions

```
1: procedure  $k$ -PREDICTEDPRIMAL-DUAL( $G, c, \hat{y}^1, \hat{y}^2, \dots, \hat{y}^k$ )
2:   for  $\ell \in [k]$  do
3:      $y^\ell \leftarrow \text{MakeFeasible}(G, c, \hat{y}^\ell)$ 
4:   end for
5:    $\ell' \leftarrow \arg \max_{\ell \in [k]} \sum_{i \in V} y_i^\ell$ 
6:    $M \leftarrow \text{Primal-Dual}(G, c, y^{\ell'})$ 
7:   Return  $M$ 
8: end procedure
```

$$\begin{aligned} \min \quad & \sum_{e \in E} c_e x_e \\ & \sum_{e \in N(i)} x_e = 1 \quad \forall i \in V \\ & x_e \geq 0 \quad \forall e \in E \end{aligned} \quad (\text{MWPM-P})$$

$$\begin{aligned} \max \quad & \sum_{i \in V} y_i \\ & y_i + y_j \leq c_e \quad \forall e = ij \in E \end{aligned} \quad (\text{MWPM-D})$$

Algorithms with Prediction Portfolios

1. Minimum Cost Bipartite Matching with Predicted Duals
- 2. Online Load Balancing with Predicted Machine Weights**
3. Scheduling with Predicted Permutations

There are m machines indexed by $[m]$ and n jobs arriving online, indexed by $[n]$.

When job j arrives, the algorithm observes:

- its size $p_j > 0$,
- a feasible set (neighborhood) $N(j) \subseteq [m]$ of machines it may be assigned to.

Predicted weights

Given weights w , the proportional weights rule **fractionally** assigns each arriving job j across its feasible machines $N(j)$ by:

$$x_{ij}(w) = \frac{w_i}{\sum_{i' \in N(j)} w_{i'}} \text{ for } i \in N(j)$$

Algorithms with Prediction Portfolios

1. Minimum Cost Bipartite Matching with Predicted Duals
- 2. Online Load Balancing with Predicted Machine Weights**
3. Scheduling with Predicted Permutations

Algorithm 3 Algorithm for combining fractional solutions online for load balancing.

```
1: procedure COMBINE-LOADBALANCING( $\beta$ )
2:    $A \leftarrow [k]$  ▷ Initially all solutions are good
3:   for each job  $j$  do
4:     Receive the assignments  $x^1, x^2, \dots, x^k$ 
5:      $A(j, \beta) \leftarrow \{\ell \in A \mid \forall i \in [m], x_{ij}^\ell > 0 \implies p_{ij} x_{ij}^\ell \leq \beta\}$ 
6:     if  $A = \emptyset$  or  $A(j, \beta) = \emptyset$  then
7:       Return “Fail”
8:     end if
9:      $\forall i \in [m], x_{ij} \leftarrow \frac{1}{|A(j, \beta)|} \sum_{\ell \in A(j, \beta)} x_{ij}^\ell$ 
10:     $B \leftarrow \{\ell \in A \mid \max_{i \in [m]} \sum_{j' \leq j} p_{ij'} x_{ij'}^\ell > \beta\}$  ▷ Bad solutions w.r.t.  $\beta$ 
11:     $A \leftarrow A \setminus B$ 
12:  end for
13: end procedure
```

Algorithms with Prediction Portfolios

Theorem 4.2. *Let $x^1, x^2, \dots, x^k$ be fractional assignments which are revealed online. If Algorithm 3 is run with $\beta := \min_{\ell \in [k]} \max_i \sum_j p_{ij} x_{ij}^\ell$, then it yields a solution of cost $O(\log k) \cdot \beta$ and never reaches the fail state (line 7 in Algorithm 3).*

Proof.

1) Decompose load by expert contributions: For any machine i , define expert ℓ 's contribution

$$C_i^\ell := \sum_j \alpha_j^\ell p_{ij} x_{ij}^{(\ell)}.$$

Then the load on machine i in the combined solution is at most

$$L_i(x) \leq \sum_{\ell=1}^k C_i^\ell.$$

Algorithms with Prediction Portfolios

Theorem 4.2. *Let $x^1, x^2, \dots, x^k$ be fractional assignments which are revealed online. If Algorithm 3 is run with $\beta := \min_{\ell \in [k]} \max_i \sum_j p_{ij} x_{ij}^\ell$, then it yields a solution of cost $O(\log k) \cdot \beta$ and never reaches the fail state (line 7 in Algorithm 3).*

2) Bound each expert by a shrinking-factor lemma:

Let j be the first time expert ℓ is removed from the active set. For jobs before j , $\alpha_{j'}^\ell \leq \frac{1}{k-\ell+1}$ and expert ℓ 's cumulative load is $\leq \beta$, contributing $\leq \frac{\beta}{k-\ell+1}$.

For the removal job j , if $\alpha_j^\ell > 0$ then the algorithm only mixes “safe” experts implying $p_{ij} x_{ij}^{(\ell)} \leq \beta$, giving another $\leq \frac{\beta}{k-\ell+1}$. After removal, $\alpha_{j'}^\ell = 0$.

Hence $C_i^\ell \leq \frac{2\beta}{k-\ell+1}$.

3) Sum over experts $\Rightarrow$ harmonic number:

$$L_i(x) \leq \sum_{\ell=1}^k \frac{2\beta}{k-\ell+1} = 2\beta \sum_{t=1}^k \frac{1}{t} = 2H_k \beta = O(\log k) \cdot \beta.$$

Algorithms with Prediction Portfolios

From samples $J_1, \dots, J_S \sim D$
predictor is close to the “gc

Theorem 4.1. *Given predicted weights $\hat{w}$, there is an online fractional algorithm which has makespan $O(\log(\eta(\hat{w}, w^*)\text{OPT}))$, where $\eta(\hat{w}, w^*) := \max_{i \in [m]} \max(\frac{\hat{w}_i}{w_i^*}, \frac{w_i^*}{\hat{w}_i})$ to be the error in the prediction.*

Multiplicative error (per-machine worst case):

$$\eta(\hat{w}, w^*) = \max_{i \in [m]} \left(\frac{\hat{w}_i}{w_i^*}, \frac{w_i^*}{\hat{w}_i} \right) \geq 1$$

Log turns error into a metric distance:

$$d(w, w') := \log \eta(w, w')$$

$$\eta(w, v) \leq \eta(w, u)\eta(u, v) \Rightarrow d(w, v) \leq d(w, u) + d(u, v)$$

so d satisfies the **triangle inequality**.

Learning objective (expected best-of- k log-error):

$$\min_{\hat{w}^{(1)}, \dots, \hat{w}^{(k)}} \mathbb{E}_{J \sim D} [\min_{\ell \in [k]} d(\hat{w}^{(\ell)}, w^*)] = \min_{\hat{w}^{(1)}, \dots, \hat{w}^{(k)}} \mathbb{E}_{J \sim D} [\min_{\ell \in [k]} \log \eta(\hat{w}^{(\ell)}, w^*)]$$

Since d is a metric, ERM over samples becomes **k -median clustering** on points w^* under distance d , enabling poly-time $O(1)$ -approx learning + poly sample complexity.

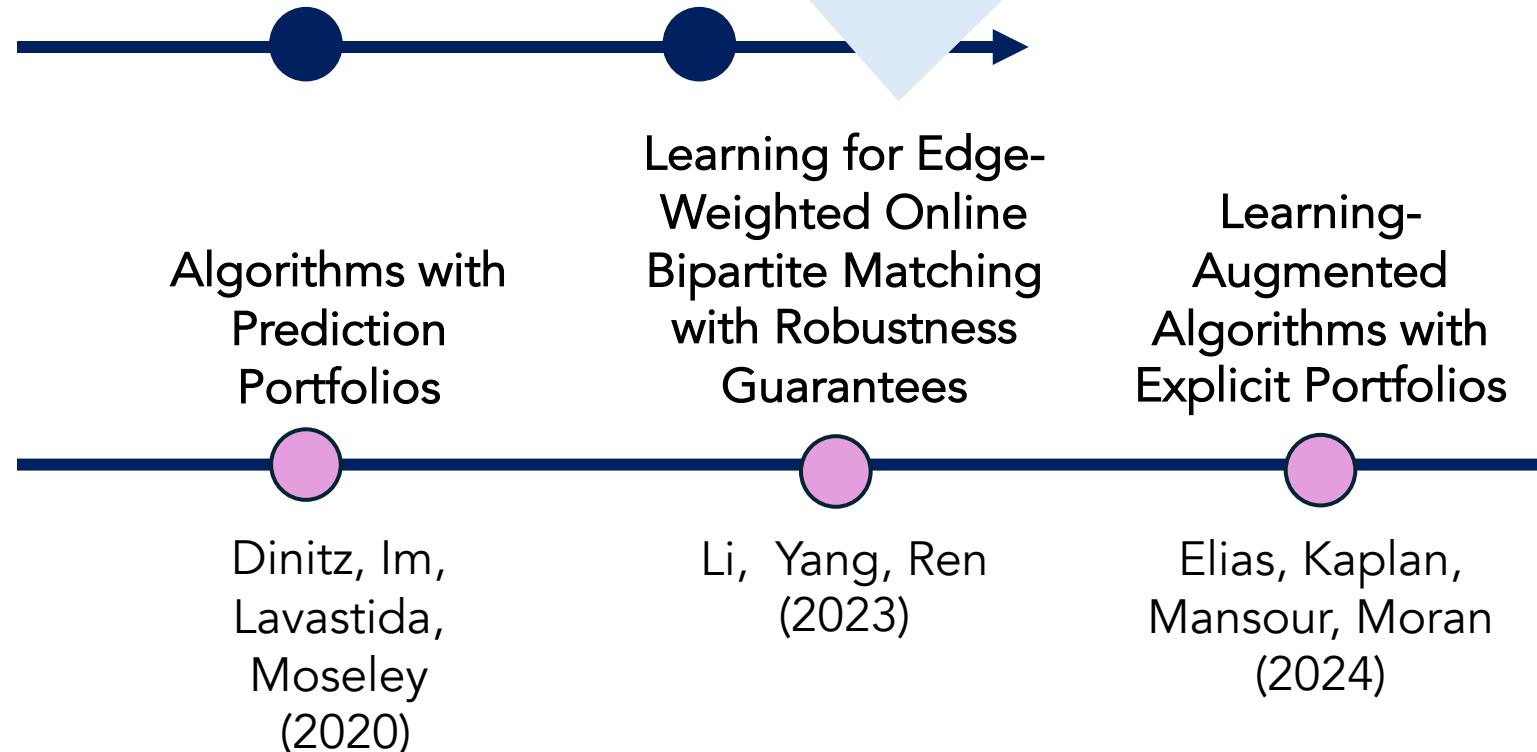
Prior work

Algorithm 1 Inference of Robust Learning-based Online Bipartite Matching (LOMAR)

Input: Competitiveness constraint $\rho \in [0, 1]$ and $B \geq 0$

```
1: for  $v = 1$  to  $|\mathcal{V}|$  do
2:   Run the expert  $\pi$  and get expert's decision  $x_v^\pi$ .
3:   If  $x_v^\pi \neq \text{skip}$ :  $\mathcal{V}_{x_v^\pi, v}^\pi = \mathcal{V}_{x_v^\pi, v-1}^\pi \cup \{v\}$ ,  
      $R_v^\pi = R_{v-1}^\pi + w_{x_v^\pi, v}$ .  
     //Update the virtual decision set  
     and reward of the expert
4:    $s_u = w_{uv} - h_\theta(I_u, w_{uv}), \forall u \in \mathcal{U}$   
     //Run RL model to get score  $s_u$  with  
     history information  $I_u$ 
5:    $\tilde{x}_v = \arg \max_{u \in \mathcal{U}_a \cup \{\text{skip}\}} \{ \{s_u\}_{u \in \mathcal{U}_a}, s_{\text{skip}} \}$ , with  
      $s_{\text{skip}} = 0$  and  $\mathcal{U}_a = \{u \in \mathcal{U} \mid |\mathcal{V}_{u, v-1}| < c_u\}$ .  
     //Get RL decision  $\tilde{x}_v$ 
6:   if Robust constraint in (3) is satisfied then
7:     Select  $x_v = \tilde{x}_v$ . //Follow RL
8:   else if  $x_v^\pi$  is available (i.e.,  $|\mathcal{V}_{x_v^\pi, v-1}| < c_{x_v^\pi}$ ) then
9:     Select  $x_v = x_v^\pi$ . //Follow the expert
10:  else
11:    Select  $x_v = \text{skip}$ .
12:  end if
13:  If  $x_v \neq \text{skip}$ ,  $\mathcal{V}_{x_v, v} = \mathcal{V}_{x_v, v-1} \cup \{v\}$ ,  
     $R_v = R_{v-1} + w_{x_v, v}$ .  
    //Update the true decision set and  
    reward
14: end for
```

The key novelty of LOMAR lies in a carefully designed online switching step that dynamically switches between the RL decision and the expert decision online, as well as a switching-aware RL Policy training algorithm.



Learning-Augmented Algorithms with Explicit Portfolios

1. Start with a **hypothesis class $\mathcal{H}$** of structured “explanations” for the instance (e.g., summaries of what the future could look like).
2. As requests/jobs arrive.
3. Predictor uses the **observed prefix** to **learn which hypothesis is most consistent** (or to produce an **improper** prediction derived from multiple hypotheses).
4. The online algorithm then **plugs that prediction in** so the performance bounds scale with how close the best hypothesis is to reality.

“In this work, we unpack the predictor and integrate the learning problem it gives rise for within the algorithmic challenge. In particular, **we allow the predictor to learn as it receives larger parts of the input**, with the ultimate goal of designing online learning algorithms specifically tailored for the algorithmic task at hand.”

Learning-Augmented Algorithms with Explicit Portfolios

Case study: **Online Load Balancing**

Predictor 3: load balancing on unrelated machines, deterministic

```
1  $A := \{1, \dots, \ell\}$ 
2 choose  $\tilde{n}_p$  as a median of  $\{n_p^i \mid i = 1, \dots, \ell\}$  for each job type  $p$  // switch to initial  $\tilde{I}$ 
3 for time step when, for some  $p$  the  $(\tilde{n}_p + 1)$ st job of type  $p$  arrives do
4    $A :=$  set of instances consistent with the input so far
5   if  $A = \emptyset$  then return ERR
6   for each job type  $p$  do
7     choose  $\tilde{n}_p$  as a median of  $\{n_p^i \mid i \in A\}$ . // switch to a new  $\tilde{I}$ 
```

Predictor 4: load balancing on unrelated machines, randomized

```
1  $A := \{1, \dots, \ell\}$  choose  $i \in A$  uniformly at random
2 set  $\tilde{n}_p = n_p^i$  for each job type  $p$  // switch to initial  $\tilde{I}$ 
3 for time step when, for some  $p$  the  $(\tilde{n}_p + 1)$ st job of type  $p$  arrives do
4    $A :=$  set of instances consistent with the input so far
5   if  $A = \emptyset$  then return ERR
6   choose  $i \in A$  uniformly at random
7   set  $\tilde{n}_p = n_p^i$  for each job type  $p$  // switch to a new  $\tilde{I}$ 
```

But now the prediction is job frequency.

Prior work

Ideally, given a hypothesis class $\mathcal{H}$, we would like a **problem-agnostic procedure** that turns $\mathcal{H}$ into a predictor.

But here the predictor is largely **co-designed with the downstream algorithm**

It is hard to transfer the technique across problems unlike Blum & Burch (2000)



Is there a more higher-level
framework to learn ideal
portfolios + selectors
adaptively as we observe
online samples?

Online Problem Formulation

Instance arrives online over T steps:

$$z = (z_1, \dots, z_T)$$

At time t , predictor observes only the **prefix**

$$z_{\leq t} = (z_1, \dots, z_t)$$

We have a **global portfolio** of κ candidates ("experts" / algorithms / parameter settings):

$$\mathcal{P} = \{\rho_1, \dots, \rho_\kappa\}$$

A **time-dependent improper predictor** can output *any* prediction derived conditioned on history.

Downstream **online algorithm** consumes $\hat{y}_t$ to choose the action at time t

$$\hat{y}_t = g_t(z_{\leq t}; \{\rho_1, \dots, \rho_\kappa\}), t \in [T],$$

where g_t is allowed to be **any** aggregation / synthesis rule over $\mathcal{P}$ (time-varying and prefix-conditioned), with output space $\hat{y}_t \in \mathcal{Y}$, and **decides**

$$\text{Alg}_t(\hat{y}_t).$$

This really looks like next token prediction.

We might need predictor g for each time step or single more powerful function that can take history as input.

This can be done from previous work

End-to-end Learning

Training set: $S = \{z^{(1)}, \dots, z^{(N)}\}$, each $z^{(i)} = (z_1^{(i)}, \dots, z_T^{(i)})$

Learn a **global portfolio** of κ candidates and a **prefix-conditioned improper predictor**

Predictor uses the observed prefix and the learned portfolio:

$$\hat{y}_t = g_t(z_{\leq t}; \mathcal{P}), t \in [T]$$

Optimize **total online cost** induced by the downstream algorithm

$$\hat{\mathcal{P}}, \hat{g} \in \arg \min_{\substack{\mathcal{P} \subseteq \mathbb{R}, |\mathcal{P}|=\kappa \\ g \in \mathcal{G}}} \frac{1}{N} \sum_{i=1}^N L(\mathcal{P}, g, z^{(i)})$$
$$L(\mathcal{P}, g, z) := \text{Cost}(\text{Alg run on } z \text{ with } \hat{y}_t = g_t(z_{\leq t}; \mathcal{P}))$$

Alternative to end-to-end learning: Can you HEDGE?

Still learn and fix portfolio $\mathcal{P}$ from the dataset.

What is your loss?

Predictor g

$$\ell_t(\hat{y}, z_{\leq t}) \approx \mathbb{E}[\text{Cost}_{t:T}(\text{Alg}|z_{\leq t}, \hat{y})|z_{\leq t}]$$

$$\ell_t(\hat{y}, z_{\leq t}) \approx \frac{1}{|\mathcal{I}(z_{\leq t})|} \sum_{i \in \mathcal{I}(z_{\leq t})} \text{Cost}_{t:T}(\text{Alg on } z^{(i)} \text{ using } \hat{y})$$

where $\mathcal{I}(z_{\leq t})$ indexes training instances with matching / similar prefix history.

This needs to be proven similarly as before for giving $\approx$ generalization guarantees.

Would this work?

Also, this is predictor-agnostic.

Do we want to favor the robust algorithm?

What if your problem is not quite like a metrical task problem?

Hedge (WMR) Algorithm

Initialize $\beta \in [0,1], w_1, w_2, \dots = 1$.

For each time step t :

1. Sample an one of κ candidates with probability $\frac{w_i}{\sum_{j=1}^{\kappa} w_j}$
2. Plug in the **proxy loss** ℓ_t
3. Reset $w_i = w_i \beta^{\ell_i}$

Possible connection to algorithmic reasoning with LLMs

1. **Transformers** are structurally fit for sequential tasks. Can we experimentally show + parameterize them so that they can sequentially choose/combine predictions/algorithms online.
 - They are exponentially many histories our algorithm should consider.
 - LLM encoders can embed them to a lower dimensional space?

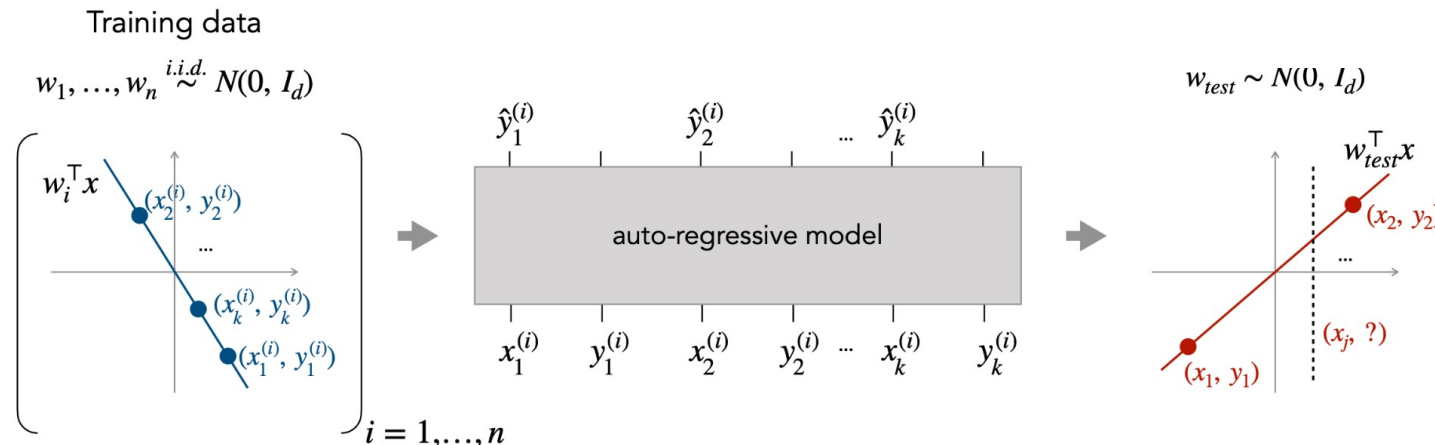
2. In context learning

TRANSFORMERS CAN DO BAYESIAN INFERENCE

Samuel Müller¹ Noah Hollmann² Sebastian Pineda¹ Josif Grabocka¹ Frank Hutter^{1,3}

¹ University of Freiburg, ² Charité Berlin, ³ Bosch Center for Artificial Intelligence

Correspondence to Samuel Müller: muellesa@cs.uni-freiburg.de



Thank you!